

Tugas 2

Struktur Data

Simple Sorting Algorithms



Dosen Pengampu :

M. Chandra Cahyo Utomo, S. Kom., M. Kom.

NIP. 199202202019031000

Disusun Oleh Kelompok 2 :

Cindy Aulia Renita	11241023
Fauzan Fayzul Haq	11241028
Thisya Darmala	11241083
Zharif Aziz Zulkarnain Widodo	11231092

27 September 2025

Dasar Teori

Algoritma *Sorting* (Pengurutan)

Algoritma *sorting* adalah suatu metode atau teknik untuk mengurutkan data atau elemen-elemen dalam suatu struktur data secara teratur. Algoritma *sorting*, bertujuan untuk mengubah data yang tidak teratur menjadi urutan yang teratur.

Jenis- Jenis Algoritma *Sorting*

1) *Bubble Sort*

Bubble Sort adalah algoritma *sorting* sederhana yang bekerja dengan cara membandingkan dan menukar pasangan elemen berdekatan secara berulang, hingga seluruh elemen tersusun dalam urutan yang benar dan tidak ada lagi penukaran yang diperlukan.

3 case *Bubble Sort* :

- 1) *Big Oh (worst case)* = $O(N^2)$
- 2) *Big Theta (average case)* = $\Theta(N^2)$
- 3) *Big Omega (best case)* = $\Omega(N)$

Bubble Sort : Lambat di semua kondisi kecuali data sudah hampir terurut.

2) *Selection Sort*

Selection sort bekerja dengan cara mencari elemen terkecil dalam daftar dan menukarnya dengan elemen pertama. Proses ini terus berlanjut untuk elemen-elemen berikutnya hingga seluruh daftar terurut.

3 case *Selection Sort* :

- 1) *Big Oh (worst case)* = $O(N^2)$
- 2) *Big Theta (average case)* = $\Theta(N^2)$
- 3) *Big Omega (best case)* = $\Omega(N^2)$

Selection Sort : Konsisten lambat karena selalu mencari elemen terkecil.

3) *Insertion Sort*

Insertion Sort membandingkan setiap elemen dengan elemen-elemen sebelumnya dan memasukkan ke dalam posisi yang benar dalam daftar terurut.

3 case *Insertion Sort* :

- 1) *Big Oh (worst case)* = $O(N^2)$
- 2) *Big Theta (average case)* = $\Theta(N^2)$
- 3) *Big Omega (best case)* = $\Omega(N)$

Insertion Sort : Sangat efisien untuk data yang sudah hampir terurut.

Source Code

No	linkedList.java
1	<code>package main.node;</code>
2	
3	<code>public class linkedlist {</code>
4	
5	<code> public node first;</code>
6	<code> private int size; // Cache size untuk performa</code>
7	
8	<code> public linkedlist() {</code>
9	<code> this.first = null;</code>
10	<code> this.size = 0;</code>
11	<code> }</code>
12	<code> public void insert(int data) {</code>
13	<code> node newNode = new node(data);</code>
14	
15	<code> if (this.first == null) {</code>
16	<code> first = newNode; // FIX: gunakan newNode, bukan</code>
17	<code>buat node baru lagi</code>
18	<code> } else {</code>
19	<code> node temp = this.first;</code>
20	<code> while (temp.getNext() != null) {</code>
21	<code> temp = temp.getNext();</code>
22	<code> }</code>
23	<code> temp.setNext(newNode);</code>
24	<code> }</code>
25	<code> size++; // Update size</code>
26	<code> }</code>
27	<code> public void insertAt(int index, int data) {</code>
28	<code> node newNode = new node(data);</code>
29	
30	<code> if (index <= 0 this.first == null) {</code>
31	<code> // Insert at the beginning</code>
32	<code> newNode.setNext(this.first);</code>
33	<code> this.first = newNode;</code>
34	<code> return;</code>
35	<code> }</code>
36	
37	<code> node temp = this.first;</code>
38	<code> int currentIndex = 0;</code>
39	
40	<code> while (temp.getNext() != null && currentIndex < index</code>
41	<code>- 1) {</code>
42	<code> temp = temp.getNext();</code>
43	<code> currentIndex++;</code>
44	<code> }</code>

```

45         newNode.setnext(temp.getnext());
46         temp.setnext(newNode);
47     }
48
49
50     // Method baru: insert di depan (lebih cepat O(1))
51     public void insertFirst(int data) {
52         node newNode = new node(data);
53         newNode.setnext(this.first);
54         this.first = newNode;
55         size++;
56     }
57
58     public boolean remove(int data) {
59         if (this.first == null) {
60             return false;
61         }
62
63         if (this.first.getdata() == data) {
64             this.first = this.first.getnext();
65             size--;
66             return true;
67         }
68
69         node temp = this.first;
70         // FIX: cek temp.getnext().getdata(), bukan loop
71         sampai akhir
72         while (temp.getnext() != null) {
73             if (temp.getnext().getdata() == data) {
74                 temp.setnext(temp.getnext().getnext());
75                 size--;
76                 return true;
77             }
78             temp = temp.getnext();
79         }
80
81         return false; // Data tidak ditemukan
82     }
83
84     // Method baru: get size yang di-cache
85     public int size() {
86         return size;
87     }
88
89     // Method baru: get node by index (untuk optimasi
90     sorting)
91     public node getNodeAt(int index) {
92         if (index < 0 || index >= size) {
93             return null;

```

92	}
93	
94	node temp = first;
95	for (int i = 0; i < index; i++) {
96	temp = temp.getnext();
97	}
98	return temp;
99	}
100	<i>// Method baru: swap data antara dua node</i>
101	public void swapData(node node1, node node2) {
102	if (node1 != null && node2 != null) {
103	int temp = node1.getdata();
104	node1.setdata(node2.getdata());
105	node2.setdata(temp);
106	}
107	}
108	
109	public void display() {
110	System.out.print("isi dari linkedlist : [");
111	node temp = this.first;
112	boolean isFirst = true;
113	while (temp != null) {
114	if (!isFirst) {
115	System.out.print(", ");
116	}
117	System.out.print(temp.getdata());
118	temp = temp.getnext();
119	isFirst = false;
120	}
121	System.out.print("]");
122	}

No	node.java
1	package main.node;
2	
3	public class node {
4	private int data;
5	private node next;
6	
7	public node(int data) {
8	this.data = data;
9	this.next = null;
10	}
11	public void setdata(int data) {

12	<code> this.data = data;</code>
13	<code> }</code>
14	
15	<code> public int getdata() {</code>
16	<code> return this.data;</code>
17	<code> }</code>
18	
19	<code> public void setnext(node next) {</code>
20	<code> this.next = next;</code>
21	<code> }</code>
22	<code> public node getnext() {</code>
23	<code> return this.next;</code>
24	<code> }</code>
25	
26	<code>}</code>

No	Alat.java
1	<code>package main.Sort;</code>
2	
3	<code>import java.util.LinkedList;</code>
4	
5	<code>public class Alat {</code>
6	<code> static void printListHighlight(LinkedList<Integer> list,</code>
7	<code>int idx1, int idx2,</code>
8	<code> String colour, String</code>
9	<code>reset) {</code>
10	<code> StringBuilder sb = new StringBuilder("[");</code>
11	<code> for (int k = 0; k < list.size(); k++) {</code>
12	<code> if (k == idx1 k == idx2) {</code>
13	<code>sb.append(colour).append(list.get(k)).append(reset);</code>
14	<code> } else {</code>
15	<code> sb.append(list.get(k));</code>
16	<code> }</code>
17	<code> if (k != list.size() - 1) sb.append(", ");</code>
18	<code> }</code>
19	<code> sb.append("]");</code>
20	<code> System.out.println(sb.toString());</code>
21	<code>}</code>
22	
23	<code> public static void timeSpent(long nanotime){</code>
24	<code> System.out.printf("%06.2f ms%n", (nanotime) /</code>
25	<code>1_000_000.0);</code>
26	<code> }</code>
27	<code>}</code>

No	Bubble.java
1	<code>package main.Sorts;</code>
2	
3	<code>import main.node.linkedlist;</code>
4	<code>import main.node.node;</code>
5	<code>import static main.Sorts.Alat.timeSpent;</code>
6	
7	<code>public class Bubble {</code>
8	<code>public static void sort(linkedlist list) {</code>
9	<code>if(list.size() > 10){</code>
10	<code>// System.out.println("huge");</code>
11	<code>hugeSort(list);</code>
12	<code>}else{</code>
13	<code>// System.out.println("Small");</code>
14	<code>smallSort(list);</code>
15	<code>}</code>
16	<code>}</code>
17	<code>public static linkedlist smallSort(linkedlist list) {</code>
18	<code>System.out.println("==== Original list =====");</code>
19	<code>displayList(list);</code>
20	
21	<code>System.out.println("\n=====");</code>
22	<code>int step = 1;</code>
23	
24	<code>// ANSI colour codes</code>
25	<code>final String YELLOW_BG = "\u001B[33m";</code>
26	<code>final String RESET = "\u001B[0m";</code>
27	
28	<code>long startTime = System.nanoTime();</code>
29	
30	<code>// OPTIMASI: Pakai array untuk akses cepat</code>
31	<code>node[] nodes = new node[list.size()];</code>
32	<code>node temp = list.first;</code>
33	<code>for (int i = 0; i < nodes.length; i++) {</code>
34	<code>nodes[i] = temp;</code>
35	<code>temp = temp.getNext();</code>
36	<code>}</code>
37	<code>// Bubble sort dengan array indexing</code>
38	<code>for (int i = 0; i < nodes.length; i++) {</code>
39	<code>boolean swapped = false;</code>
40	<code>for (int j = 0; j < nodes.length - i - 1; j++) {</code>
41	<code>int first = nodes[j].getData();</code>
42	<code>int second = nodes[j + 1].getData();</code>
43	<code>if (first > second) {</code>
44	<code></code>

```

45         System.out.println("Step " + step++ + ":
46 Swapping " + first + " and " + second);
47
48         // Swap data langsung
49         nodes[j].setdata(second);
50         nodes[j + 1].setdata(first);
51         swapped = true;
52
53         printListHighlight(list, j, j + 1,
54 YELLOW_BG, RESET);
55         System.out.println();
56     }
57 }
58
59     if (!swapped) {
60         break; // Sudah sorted
61     }
62
63     timeSpent(System.nanoTime() - startTime);
64     return list;
65 }
66
67 public static linkedlist hugeSort(linkedlist list) {
68     long startTime = System.nanoTime();
69
70     if (list.size() <= 1) {
71         timeSpent(System.nanoTime() - startTime);
72         return list;
73     }
74
75     // OPTIMASI: Array untuk akses O(1)
76     node[] nodes = new node[list.size()];
77     node temp = list.first;
78     for (int i = 0; i < nodes.length; i++) {
79         nodes[i] = temp;
80         temp = temp.getnext();
81     }
82
83     // Bubble sort super cepat
84     for (int i = 0; i < nodes.length; i++) {
85         boolean swapped = false;
86
87         for (int j = 0; j < nodes.length - i - 1; j++) {
88             if (nodes[j].getdata() > nodes[j +
89 1].getdata()) {
90                 // Swap data
91                 int tempData = nodes[j].getdata();
92                 nodes[j].setdata(nodes[j + 1].getdata());
93                 nodes[j + 1].setdata(tempData);

```



```

92         swapped = true;
93     }
94 }
95
96     if (!swapped) {
97         break;
98     }
99 }
100
101     timeSpent(System.nanoTime() - startTime);
102     return list;
103 }
104
105 // Helper methods
106 private static void displayList(linkedlist list) {
107     System.out.print("[");
108     node temp = list.first;
109     boolean first = true;
110     while (temp != null) {
111         if (!first) {
112             System.out.print(", ");
113         }
114         System.out.print(temp.getData());
115         temp = temp.getNext();
116         first = false;
117     }
118     System.out.print("]");
119 }
120
121 private static void printListHighlight(linkedlist list,
122 int pos1, int pos2, String colorCode, String resetCode) {
123     System.out.print("[");
124     node temp = list.first;
125     int index = 0;
126     boolean first = true;
127
128     while (temp != null) {
129         if (!first) {
130             System.out.print(", ");
131         }
132
133         if (index == pos1 || index == pos2) {
134             System.out.print(colorCode + temp.getData() +
135 resetCode);
136         } else {
137             System.out.print(temp.getData());
138         }
139
140         temp = temp.getNext();
141         index++;
142     }
143     System.out.print("]");
144 }

```

139	first = false;
140	}
141	System.out.print("]");
142	}
	}

No	Selection.java
1	package main.Sorts;
2	
3	import main.node.linkedlist;
4	import main.node.node;
5	import static main.Sorts.Alat.timeSpent;
6	
7	public class Selection {
8	public static void sort(linkedlist list) {
9	if(list.size() > 10){
10	hugeSort(list);
11	}else{
12	smallSort(list);
13	}
14	}
15	public static linkedlist smallSort(linkedlist list) {
16	System.out.println("==== Original list =====");
17	displayList(list);
18	
19	System.out.println("\n=====");
20	int step = 1;
21	
22	// ANSI colour codes
23	final String YELLOW_BG = "\u001B[33m";
24	final String RESET = "\u001B[0m";
25	
26	long startTime = System.nanoTime();
27	
28	if (list.size() <= 1) {
29	timeSpent(System.nanoTime() - startTime);
30	return list;
31	}
32	// OPTIMASI: Pakai array untuk akses cepat
33	node[] nodes = new node[list.size()];
34	node temp = list.first;
35	for (int i = 0; i < nodes.length; i++) {
36	nodes[i] = temp;
37	temp = temp.getnext();
	}

```

38
39     // Selection sort dengan array indexing
40     for (int i = 0; i < nodes.length - 1; i++) {
41         int minIndex = i;
42
43         // Cari minimum dari sisa array
44         for (int j = i + 1; j < nodes.length; j++) {
45             if (nodes[j].getdata() <
46 nodes[minIndex].getdata()) {
47                 minIndex = j;
48             }
49
50             // Tukar jika perlu
51             if (minIndex != i) {
52                 System.out.println("Step " + step++ + ": Min
53 val " + nodes[minIndex].getdata());
54
55                 // Tukar data
56                 int tempData = nodes[i].getdata();
57                 nodes[i].setdata(nodes[minIndex].getdata());
58                 nodes[minIndex].setdata(tempData);
59
60                 printListHighlight(list, i, -1, YELLOW_BG,
61 RESET);
62                 System.out.println();
63             } else {
64                 System.out.println("Step " + step++ + ":
65 Element " + nodes[i].getdata() + " already low valued");
66                 printListHighlight(list, i, -1, YELLOW_BG,
67 RESET);
68                 System.out.println();
69             }
70
71             }
72
73             timeSpent(System.nanoTime() - startTime);
74             return list;
75         }
76
77         public static linkedlist hugeSort(linkedlist list) {
78             long startTime = System.nanoTime();
79
80             if (list.size() <= 1) {
81                 timeSpent(System.nanoTime() - startTime);
82                 return list;
83             }
84
85             // OPTIMASI: Array untuk akses O(1)
86             node[] nodes = new node[list.size()];
87             node temp = list.first;

```

```

84         for (int i = 0; i < nodes.length; i++) {
85             nodes[i] = temp;
86             temp = temp.getnext();
87         }
88
89         // Selection sort super cepat
90         for (int i = 0; i < nodes.length - 1; i++) {
91             int minIndex = i;
92
93             for (int j = i + 1; j < nodes.length; j++) {
94                 if (nodes[j].getdata() <
nodes[minIndex].getdata()) {
95                     minIndex = j;
96                 }
97             }
98
99             if (minIndex != i) {
100                 int tempData = nodes[i].getdata();
101                 nodes[i].setdata(nodes[minIndex].getdata());
102                 nodes[minIndex].setdata(tempData);
103             }
104         }
105
106         timeSpent(System.nanoTime() - startTime);
107         return list;
108     }
109
110     // Helper methods
111     private static void displayList(linkedlist list) {
112         System.out.print("[");
113         node temp = list.first;
114         boolean first = true;
115         while (temp != null) {
116             if (!first) {
117                 System.out.print(", ");
118             }
119             System.out.print(temp.getdata());
120             temp = temp.getnext();
121             first = false;
122         }
123         System.out.print("]");
124     }
125
126     private static void printListHighlight(linkedlist list,
int pos1, int pos2, String colorCode, String resetCode) {
127         System.out.print("[");
128         node temp = list.first;
129         int index = 0;
130         boolean first = true;

```

```

131         while (temp != null) {
132             if (!first) {
133                 System.out.print(", ");
134             }
135             if (index == pos1 || index == pos2) {
136                 System.out.print(colorCode + temp.getdata() +
137 resetCode);
138             } else {
139                 System.out.print(temp.getdata());
140             }
141             temp = temp.getNext();
142             index++;
143             first = false;
144         }
145         System.out.print("]");
146     }
147 }
148 }
149

```

No	Insertion.java
1	<code>package main.Sorts;</code>
2	
3	<code>import main.node.linkedlist;</code>
4	<code>import main.node.node;</code>
5	<code>import static main.Sorts.Alat.timeSpent;</code>
6	
7	<code>public class Insertion {</code>
8	<code>public static void sort(linkedlist list) {</code>
9	<code>if(list.size() > 10){</code>
10	<code>hugeSort(list);</code>
11	<code>}else{</code>
12	<code>smallSort(list);</code>
13	<code>}</code>
14	<code>}</code>
15	<code>public static linkedlist smallSort(linkedlist list) {</code>
16	<code>System.out.println("==== Original list =====");</code>
17	<code>displayList(list);</code>
18	
19	<code>System.out.println("\n=====");</code>
20	<code>int step = 1;</code>
21	
22	<code>// ANSI colour codes</code>
23	<code>final String YELLOW_BG = "\u001B[33m";</code>
24	<code>final String RESET = "\u001B[0m";</code>

```

25
26         long startTime = System.nanoTime();
27
28         if (list.first == null || list.first.getNext() ==
29 null) {
30             timeSpent(System.nanoTime() - startTime);
31             return list;
32         }
33
34         // OPTIMASI: Pakai array untuk akses cepat ke node
35         node[] nodes = new node[list.size()];
36         node temp = list.first;
37         for (int i = 0; i < nodes.length; i++) {
38             nodes[i] = temp;
39             temp = temp.getNext();
40         }
41
42         // Insertion sort dengan array indexing
43         for (int i = 1; i < nodes.length; i++) {
44             int key = nodes[i].getData();
45             int j = i - 1;
46
47             System.out.println("Step " + step++ + ": Arrange "
48 + key);
49
50             // Geser elemen yang lebih besar ke kanan
51             while (j >= 0 && nodes[j].getData() > key) {
52                 nodes[j + 1].setData(nodes[j].getData());
53                 j--;
54             }
55             nodes[j + 1].setData(key);
56
57             printListHighlight(list, j + 1, -1, YELLOW_BG,
58 RESET);
59             System.out.println();
60         }
61
62         timeSpent(System.nanoTime() - startTime);
63         return list;
64     }
65
66     public static linkedlist hugeSort(linkedlist list) {
67         long startTime = System.nanoTime();
68
69         if (list.first == null || list.first.getNext() ==
70 null) {
71             timeSpent(System.nanoTime() - startTime);
72             return list;
73         }

```

```

72      // OPTIMASI: Pakai array untuk akses O(1)
73      node[] nodes = new node[list.size()];
74      node temp = list.first;
75      for (int i = 0; i < nodes.length; i++) {
76          nodes[i] = temp;
77          temp = temp.getnext();
78      }
79
80      // Insertion sort super cepat
81      for (int i = 1; i < nodes.length; i++) {
82          int key = nodes[i].getdata();
83          int j = i - 1;
84
85          while (j >= 0 && nodes[j].getdata() > key) {
86              nodes[j + 1].setdata(nodes[j].getdata());
87              j--;
88          }
89          nodes[j + 1].setdata(key);
90
91      }
92      timeSpent(System.nanoTime() - startTime);
93      return list;
94  }
95
96  // Helper methods
97  private static void displayList(linkedlist list) {
98      System.out.print("[");
99      node temp = list.first;
100     boolean first = true;
101     while (temp != null) {
102         if (!first) {
103             System.out.print(", ");
104         }
105         System.out.print(temp.getdata());
106         temp = temp.getnext();
107         first = false;
108     }
109     System.out.print("]");
110 }
111
112 private static void printListHighlight(linkedlist list,
113 int pos1, int pos2, String colorCode, String resetCode) {
114     System.out.print("[");
115     node temp = list.first;
116     int index = 0;
117     boolean first = true;
118
119     while (temp != null) {
120         if (!first) {
121             System.out.print(", ");

```

```

119         }
120
121         if (index == pos1 || index == pos2) {
122             System.out.print(colorCode + temp.getdata() +
123 resetCode);
124         } else {
125             System.out.print(temp.getdata());
126         }
127
128         temp = temp.getNext();
129         index++;
130         first = false;
131     }
132     System.out.print("]");
133 }
134

```

No	Main.java
1	<code>package main;</code>
2	
3	<code>import main.Sorts.Bubble;</code>
4	<code>import main.Sorts.Selection;</code>
5	<code>import main.Sorts.Insertion;</code>
6	<code>import main.node.linkedlist;</code>
7	<code>import main.node.node;</code>
8	
9	<code>import java.util.Random;</code>
10	<code>import java.util.Scanner;</code>
11	<code>public class Main {</code>
12	<code>public static void main(String[] args) {</code>
13	<code>Scanner input = new Scanner(System.in);</code>
14	<code>Random rand = new Random();</code>
15	
16	<code>System.out.println("Masukkan banyaknya nilai dalam</code>
17	<code>list!:\n jika nilai > 10, akan otomatis melakukan test</code>
18	<code>kecepatan!");</code>
19	<code>int jumlah_perulangan = input.nextInt();</code>
20	
21	<code>System.out.println("Apakah nilai dalam list dapat</code>
22	<code>duplicate? [y/n]: ");</code>
23	<code>String answer = input.next().trim().toLowerCase();</code>
24	<code>Boolean isDupeable = answer.equals("y");</code>
25	
26	<code>System.out.println("====> Membuat angka random!");</code>


```

27         linkedlist numbers = new linkedlist();
28         if(isDupeable){
29             dupes(numbers, rand, jumlah_perulangan);
30         }else{
31             noDupes(numbers, rand, jumlah_perulangan);
32         }
33         linkedlist numbers1 = copyLinkedList(numbers);
34         linkedlist numbers2 = copyLinkedList(numbers);
35         linkedlist numbers3 = copyLinkedList(numbers);
36         System.out.println("====> Angka random berhasil
37         dibuat");
38
39         if (jumlah_perulangan <= 10){
40             Scanner option = new Scanner(System.in);
41             System.out.println("=====\n1. Bubble
42             sort\n2. Selection sort\n3. Insertion
43             sort\n=====");
44             System.out.print("Pilih sort method: ");
45             int sortChoise = option.nextInt();
46
47             System.out.println("-----
48             -----");
49
50             switch (sortChoise) {
51                 case 1:
52                     Bubble.sort(numbers1);
53                     break;
54                 case 2:
55                     Selection.sort(numbers2);
56                     break;
57                 case 3:
58                     Insertion.sort(numbers3);
59                     break;
60                 default:
61                     System.out.println("Whooops salah nomor
62                     jadi pake bubble aja");
63                     Bubble.sort(numbers);
64             }
65             return;
66
67             System.out.println("\n====> Test sort's
68             speed\n-----");
69             System.out.print("Bubble Sort      :");
70             Bubble.sort(numbers1);
71             System.out.print("Selection sort   :");
72             Selection.sort(numbers2);
73             System.out.print("Insertion sort    :");
74             Insertion.sort(numbers3);
75             System.out.println("-----");

```

```

74     }
75
76     // Method to check if linkedlist contains a value
77     public static boolean contains(linkedlist list, int value)
78     {
79         node temp = list.first;
80         while (temp != null) {
81             if (temp.getdata() == value) {
82                 return true;
83             }
84             temp = temp.getnext();
85         }
86         return false;
87     }
88
89     public static linkedlist noDups(linkedlist list, Random
90     rand, int jumlah) {
91         for (int i = 0; i < jumlah; i++) {
92             int randomIndex = rand.nextInt(i + 1); // between
93             0 and i inclusive
94             list.insertAt(randomIndex, i); // Insert at random
95             index
96         }
97         return list;
98     }
99
100     public static void dups(linkedlist list, Random rand, int
101     jumlah){
102         for (int i = 0; i < jumlah; i++) {
103             list.insert(rand.nextInt(jumlah - 1)); // Reduced
104             range for better visualization
105         }
106     }
107
108     // Helper method to copy a linkedlist
109     public static linkedlist copyLinkedList(linkedlist
110     original) {
111         linkedlist copy = new linkedlist();
112         node temp = original.first;
113         while (temp != null) {
114             copy.insert(temp.getdata());
115             temp = temp.getnext();
116         }
117         return copy;
118     }
119 }

```

Screenshot

1. Bubble sort bekerja dengan membandingkan dua elemen berdekatan dan menukarnya jika urutannya salah, sehingga angka besar “mengendap” ke belakang dan angka kecil “mengambang” ke depan seperti gelembung. Algoritma melakukan banyak pertukaran (swap) untuk menempatkan angka terkecil di posisi awal, dimulai dari list acak [2, 4, 6, 8, 3, 1, 5, 7, 9, 0] hingga menjadi [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], dengan total 25 langkah dan waktu eksekusi sekitar 017,06 ms.

```
Masukkan banyaknya nilai dalam list!:
jika nilai > 10, akan otomatis melakukan test kecepatan!
10
Apakah nilai dalam list dapat duplicate? [y/n]:
n
====> Membuat angka random!
====> Angka random berhasil dibuat
=====
1. Bubble sort
2. Selection sort
3. Insertion sort
=====
Pilih sort method: 1
-----
===== Original list =====
[5, 7, 2, 0, 9, 8, 3, 4, 6, 1]
=====
Step 1: Swapping 7 and 2
[5, 2, 7, 0, 9, 8, 3, 4, 6, 1]
Step 2: Swapping 7 and 0
[5, 2, 0, 7, 9, 8, 3, 4, 6, 1]
Step 3: Swapping 9 and 8
[5, 2, 0, 7, 8, 9, 3, 4, 6, 1]
Step 4: Swapping 9 and 3
[5, 2, 0, 7, 8, 3, 9, 4, 6, 1]
Step 5: Swapping 9 and 4
[5, 2, 0, 7, 8, 3, 4, 9, 6, 1]
Step 6: Swapping 9 and 6
[5, 2, 0, 7, 8, 3, 4, 6, 9, 1]
Step 7: Swapping 9 and 1
[5, 2, 0, 7, 8, 3, 4, 6, 1, 9]
Step 8: Swapping 5 and 2
[2, 5, 0, 7, 8, 3, 4, 6, 1, 9]
Step 9: Swapping 5 and 0
[2, 0, 5, 7, 8, 3, 4, 6, 1, 9]
Step 10: Swapping 8 and 3
[2, 0, 5, 7, 3, 8, 4, 6, 1, 9]
```

```
Step 11: Swapping 8 and 4
[2, 0, 5, 7, 3, 4, 8, 6, 1, 9]
Step 12: Swapping 8 and 6
[2, 0, 5, 7, 3, 4, 6, 8, 1, 9]
Step 13: Swapping 8 and 1
[2, 0, 5, 7, 3, 4, 6, 1, 8, 9]
Step 14: Swapping 2 and 0
[0, 2, 5, 7, 3, 4, 6, 1, 8, 9]
Step 15: Swapping 7 and 3
[0, 2, 5, 3, 7, 4, 6, 1, 8, 9]
Step 16: Swapping 7 and 4
[0, 2, 5, 3, 4, 7, 6, 1, 8, 9]
Step 17: Swapping 7 and 6
[0, 2, 5, 3, 4, 6, 7, 1, 8, 9]
Step 18: Swapping 7 and 1
[0, 2, 5, 3, 4, 6, 1, 7, 8, 9]
Step 19: Swapping 5 and 3
[0, 2, 3, 5, 4, 6, 1, 7, 8, 9]
Step 20: Swapping 5 and 4
[0, 2, 3, 4, 5, 6, 1, 7, 8, 9]
Step 21: Swapping 6 and 1
[0, 2, 3, 4, 5, 1, 6, 7, 8, 9]
Step 22: Swapping 5 and 1
[0, 2, 3, 4, 1, 5, 6, 7, 8, 9]
Step 23: Swapping 4 and 1
[0, 2, 3, 1, 4, 5, 6, 7, 8, 9]
Step 24: Swapping 3 and 1
[0, 2, 1, 3, 4, 5, 6, 7, 8, 9]
Step 25: Swapping 2 and 1
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
017,06 ms
```

2. Selection sort bekerja dengan cara memilih elemen terkecil dari bagian list yang belum terurut, lalu menukarnya dengan elemen di posisi awal bagian tersebut. List diurutkan dengan mencari indeks nilai minimum di setiap langkah misalnya pada Step 1 ditemukan nilai terkecil di indeks ke-5 (yaitu 0), lalu ditukar ke posisi pertama. Proses ini diulang dari kiri ke kanan, sehingga setiap elemen ditempatkan di posisi yang benar satu per satu. Hasil akhirnya adalah list dengan waktu eksekusi sekitar 016,77 ms.

```

Masukkan banyaknya nilai dalam list!:
jika nilai > 10, akan otomatis melakukan test kecepatan!
10
Apakah nilai dalam list dapat duplicate? [y/n]:
n
====> Membuat angka random!
====> Angka random berhasil dibuat
=====
1. Bubble sort
2. Selection sort
3. Insertion sort
=====
Pilih sort method: 2
-----
===== Original list =====
[4, 2, 5, 9, 0, 3, 6, 1, 7, 8]
=====
Step 1: Min val 0
[0, 2, 5, 9, 4, 3, 6, 1, 7, 8]
Step 2: Min val 1
[0, 1, 5, 9, 4, 3, 6, 2, 7, 8]
Step 3: Min val 2
[0, 1, 2, 9, 4, 3, 6, 5, 7, 8]
Step 4: Min val 3
[0, 1, 2, 3, 4, 9, 6, 5, 7, 8]
Step 5: Element 4 already low valued
[0, 1, 2, 3, 4, 9, 6, 5, 7, 8]
Step 6: Min val 5
[0, 1, 2, 3, 4, 5, 6, 9, 7, 8]
Step 7: Element 6 already low valued
[0, 1, 2, 3, 4, 5, 6, 9, 7, 8]
Step 8: Min val 7
[0, 1, 2, 3, 4, 5, 6, 7, 9, 8]
Step 9: Min val 8
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
016,77 ms

```

3. Insertion sort proses kerja algoritma insertion sort dalam sebuah program yang mengurutkan 10 angka acak tanpa duplikat. Setelah pengguna memilih metode ke-3 (insertion sort), program menampilkan daftar angka asli lalu menyusun ulang angka satu per satu dengan menyisipkan elemen ke posisi yang tepat di bagian yang sudah terurut. Setiap langkah menunjukkan perubahan posisi angka hingga akhirnya menghasilkan daftar yang terurut sempurna dari 0 hingga 9. Proses ini selesai dalam waktu 007,26 ms.

```
jika nilai > 10, akan otomatis melakukan test kecepatan!
```

```
10
```

```
Apakah nilai dalam list dapat duplicate? [y/n]:
```

```
n
```

```
=====> Membuat angka random!
```

```
=====> Angka random berhasil dibuat
```

```
=====
```

```
1. Bubble sort
```

```
2. Selection sort
```

```
3. Insertion sort
```

```
=====
```

```
Pilih sort method: 3
```

```
-----
```

```
===== Original list =====
```

```
[3, 8, 4, 9, 1, 5, 7, 0, 6, 2]
```

```
=====
```

```
Step 1: Arrange 8
```

```
[3, 8, 4, 9, 1, 5, 7, 0, 6, 2]
```

```
Step 2: Arrange 4
```

```
[3, 4, 8, 9, 1, 5, 7, 0, 6, 2]
```

```
Step 3: Arrange 9
```

```
[3, 4, 8, 9, 1, 5, 7, 0, 6, 2]
```

```
Step 4: Arrange 1
```

```
[1, 3, 4, 8, 9, 5, 7, 0, 6, 2]
```

```
Step 5: Arrange 5
```

```
[1, 3, 4, 5, 8, 9, 7, 0, 6, 2]
```

```
Step 6: Arrange 7
```

```
[1, 3, 4, 5, 7, 8, 9, 0, 6, 2]
```

```
Step 7: Arrange 0
```

```
[0, 1, 3, 4, 5, 7, 8, 9, 6, 2]
```

```
Step 8: Arrange 6
```

```
[0, 1, 3, 4, 5, 6, 7, 8, 9, 2]
```

```
Step 9: Arrange 2
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
007,26 ms
```

4. Insertion Sort jauh lebih efisien dibandingkan Bubble Sort dan Selection Sort untuk jumlah data besar (100.000 elemen tanpa duplikat). Waktu eksekusi Insertion Sort hanya 4.482,65 ms, sedangkan Selection Sort memakan waktu 10003,06 ms, dan Bubble Sort bahkan lebih lambat dengan 134226,94 ms. Ini menunjukkan bahwa meskipun ketiganya termasuk algoritma sederhana dengan kompleksitas waktu rata-rata $O(n^2)$, Insertion Sort memiliki performa yang lebih baik dalam praktik, terutama ketika data memiliki pola tertentu atau sebagian sudah terurut.

```
Masukkan banyaknya nilai dalam list!:
```

```
jika nilai > 10, akan otomatis melakukan test kecepatan!
```

```
100000
```

```
Apakah nilai dalam list dapat duplicate? [y/n]:
```

```
n
```

```
=====> Membuat angka random!
```

```
=====> Angka random berhasil dibuat
```

```
=====> Test sort's speed
```

```
-----
```

```
Bubble Sort      :134226,94 ms
```

```
Selection sort   :10003,06 ms
```

```
Insertion sort   :4482,65 ms
```

```
-----
```

Refrensi

- Kusumaningrum, A. (2021). *Perbandingan Kecepatan antara Selection Sort, Insertion Sort, dan Bubble Sort*. AMIK BSI Yogyakarta. Diakses dari <https://ejournal.unjaya.ac.id>
- Mahrozi, N., & Faisal, M. (2023). *Analisis Perbandingan Kecepatan Algoritma Selection Sort dan Bubble Sort*. Universitas Islam Negeri Maulana Malik Ibrahim Malang. Diakses dari <http://repository.uin-malang.ac.id>
- Annisa. (2023). *Pengertian Algoritma Sorting (Pengurutan) Dalam Pemrograman*. Fakultas Ilmu Komputer dan Teknologi Informasi, Universitas Muhammadiyah Sumatera Utara. Diakses dari <https://fikti.umsu.ac.id/pengertian-algoritma-sorting-pengurutan-dalam-pemrograman/>

Repository

<https://github.com/ZAZW27/3SortDiff>